

An allocator-aware optional type

Abstract

Library types that can potentially hold *allocator-aware* (AA) objects should, themselves, be allocator-aware. A PMR container, for example, depends on AA types following a known AA protocol so that it (the container) can uniformly manage its memory. Even types that don't manage their own memory, such as `tuple`, follow the AA rules when they hold one more more AA elements. (A special case is `pair`, which is not actually AA but effectively follows the rules through special handling in *uses-allocator construction*.)

The current definition of `std::optional` does not follow the rules for an AA type, even when holding an AA value. This limitation makes `std::optional` unsuitable for storage in an AA container when memory allocation customization is needed.

In this paper, we propose a new, allocator-aware `std::basic_optional` usable as a container element type of allocator aware containers, and in any other context where allocator propagation is expected. This new type would not replace the current `std::optional` as the desired behaviour is not compatible with how `std::optional` handles allocator aware types at the moment. We do propose having a special treatment for `std::basic_optional` and `std::optional` that allows for certain implicit conversions between the two types. We also propose an `std::pmr::optional` type which is a specialisation of `std::basic_optional` for `std::pmr::polymorphic_allocator<>`.

This is a complete proposal with formal wording.

change history :

- replaced `alloc_optional` with additional overloads of `make_optional` that take an `allocator_arg_t` : it was observed that `alloc_optional` doesn't actually do any allocation, and that the name is not appropriate
 - fixed the order of arguments in the *Returns* clause of what used to be `alloc_optional`
 - adjusted for LWG 2833
 - added discussion on deduction guides
 - removed mentions of using a type alias since the type alias approach has proven to be problematic
 - modified the specification of `std::pmr::optional` to cover both AA and non AA types
 - added CTAD specification
 - modified the specification of `std::pmr::optional` to be expressed in terms of `std::pmr::polymorphic_allocator<>`, as opposed to `std::pmr::memory_resource`
 - moved `std::pmr::optional` to `optional` header as discussed in the reflector discussion
 - added request for discussion on making `std::pmr::optional` a more allocator generic type.
 - added free swap function to be in line with P0178
- R3:
- replaced instances of `M_alloc` with `alloc`
 - clarified `pmr::optional` constructor descriptions to call out *uses-allocator construction*
 - expanded `basic_optional` discussion
- R4:
- switched to `std::pmr::optional` being a specialisation of `std::basic_optional`
 - modified the wording to cover `std::basic_optional`
- R5:
- various clean up of proposed wording
 - setting the default allocator to be `remove_cv_t` version of `value_type`
 - removed `make` facilities

Motivation and Proposal Summary

C++ has been moving towards a consistent PMR allocator model whereby *allocator-aware* (AA) types adhere to a few rules:

1. Once constructed, an object's allocator never changes.
2. The object's constructors all follow one of two argument protocols so that they work with *uses-allocator construction* and thus can be inserted as container elements using the container's allocator.
3. An object that can contain an AA element of user-specified type should itself be AA and use its allocator to initialize its AA subobjects.

The current `std::optional` does not follow the above rules. When disengaged, it forgets its allocator, violating the first rule; its constructors don't follow the AA protocols, so containers cannot pass their allocators to `optional` elements; and it does not hold on an allocator by which it can initialize its contained object. As a result, `std::optional` is not a good fit for situations where allocators are used. A `std::pmr::vector<optional<std::pmr::string>>`, for example, cannot ensure that all of strings within it use the same allocator, violating a key invariant of PMR containers. If one of the elements of the vector became disengaged and re-engaged, for example, the allocator for that one element could change.

The `basic_optional` class template proposed here properly adheres to the rules of an AA type when instantiated with an AA value type. It holds a copy of the allocator used to construct it, even when disengaged, and uses that allocator to construct its element when re-engaged. This allows containers of `basic_optional` objects to correctly manage their memory. Furthermore, to support allocators with different allocator traits to the PMR allocator model, `basic_optional` also uses allocator propagation traits to determine the behaviour of optional object with regards to allocator propagation.

Design decisions:

basic_optional supports non-scoped propagating allocators

There are two ways of viewing `basic_optional<T>` from allocator propagation perspective :

#1 `basic_optional<T>` is like a `std::tuple`, i.e. it only accepts the allocator at construction so it can forward it to the `value_type` object. One can use a non-scoped propagation allocator, and when using a scoped propagation allocator `basic_optional<T>` will not "consume" an allocator level. An optional object is in a way like a tuple object as it does not use the allocator itself, it only passes it into the `value_type` object.

#2 `basic_optional<T>` is like a `std::vector`, i.e. it is a container of one or zero elements, and one should use a scoped propagating allocator if one wants the `value_type` object to use the allocator. In this approach `basic_optional<T>` will "consume" an allocator level. Using non-scoped propagating allocators makes little sense in this scenario.

The proposal implements #1 as `basic_optional` itself does not allocate any memory so it makes little sense for it to consume an allocator level.

The basic design of an AA optional is straight-forward: Add an allocator to all of its constructors and use that allocator to construct the value object each type the optional is engaged. However, when holding a non-AA type, there is no reason to pay the overhead of storing an allocator.

`uses_allocator<basic_optional<T,Alloc>>` corresponds to `uses_allocator<T,Alloc>`. We believe there is no need to support the AA constructor interface for non-AA types. Generic programming should use `uses_allocator` construction and `std::make_obj_using_allocator` to invoke the correct constructors.

Conversions between `std::optional<T>` and `std::basic_optional<T,Alloc>`

Consider:

```
basic_optional<T,Alloc> x;
optional<T> y = x; // #1
basic_optional<T,Alloc> z = y; // #2

optional<T> foo_constref(const optional<T>& );
foo_constref(x); // #3

void foo_ref(optional<T>&);
foo_ref(x) // #4
```

In the example above, we do not believe #1,#2, and #3 are ever problematic, but may be useful for code which currently uses `optional`. However, #4, if allowed, would potentially modify the `basic_optional` in a way that does not preserve the allocator requirements. Note that #4 is only problematic if `uses_allocator<basic_optional<T,Alloc>> == true`.

Allowing #4 for cases where `uses_allocator<basic_optional<T,Alloc>> == false` would make re-using codebases which traffic in non allocator aware `optional` possible when allocator does not matter. However, it adds to the complexity of design. It is also questionable whether there is a need for this conversion. One can have two reasons to use `basic_optional` with non allocator aware types:

- writing generic code which serves both allocator and non allocator aware types. Allowing interoperability with `optional` for only certain cases seems unhelpful in such a case.

- using `basic_optional` for all optional types for simplicity purposes. Allowing interoperability with `optional` would be useful in this case.

We propose to not implement conversion #4 until the time it is needed. However, library implementors might want to consider this as a possible extension because it might inform the implementation design they go for.

Allocator used in `value_or`

It is not all that obvious which allocator should be used for the object returned by `value_or`. Should the decision be left to the type or should it be mandated by the `optional` ? That is, should the object be constructed by plain copy/move construction or with `uses_allocator` construction? The proposal leaves the decision to the `value_type`. If the user cares about the allocator of the returned object, it should be explicitly provided by the user. We may consider providing allocator extended version of `value_or` in the future, if this use case proves to be common enough.

Deduction guides for mixed optionals unpack

Consider

```
std::optional x1 = value_type();
std::basic_optional copy1 = x1; // #1

std::optional x2 = value_type_aa();
std::basic_optional copy2 = x2; // #2

std::basic_optional x3 = value_type();
std::optional copy3 = x3; // #3

std::basic_optional x4 = value_type_aa();
std::optional copy4 = x4; // #4
```

What should the types of `x1`, `x2`, `x3`, and `x4` in the above example be ?

The current proposal favours unpacking, and the types are:

```
#1 std::basic_optional<value_type, allocator<value_type>>,
#2 std::basic_optional<value_type_aa, allocator<value_type_aa>>,
#3 std::optional<value_type>, and
#4 std::optional<value_type_aa>.
```

This seems to fit with the idea that `std::basic_optional` and `std::optional` are types that are close enough that they deserve to be treated equally wherever they can. However, it does mean that they are somehow blessed by the standard to work together in a way a user defined type which inherits from `std::optional` wouldn't be.

Non allocator aware `basic_optional` is not an alias for `optional`

An early draft of this proposal suggested using a type alias where a non-AA `basic_optional` aliases `std::optional`, and AA `basic_optional` aliases a new unnamed type. However, this causes usability issues. Type deduction in cases like :

```
template <typename T>
void foo(basic_optional<T>);
```

did not work. The above was equivalent to

```
template <typename T, typename Alloc>
void foo(std::conditional<std::uses_allocator<T, Alloc>>::value,
        __basic_optional<T>,
        std::optional<T>>::type);
```

and using the nested type of the `std::conditional` as the function template parameter made for an undeduced context.

Free swap has a wide contract

The free swap function has been made to have a wide contract and work with different allocators, as discussed in P0178 (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0178r0.html>)

Make facilities are not provided

Make facilities have been deemed unnecessary with the availability of CTAD. The current version of the paper doesn't propose `std::make_basic_optional` nor `std::pmr::make_optional`. Previous version suggested those be included, but with implementation experience, we found a complexity in allowing the allocator type to be both specifiable and defaulted in non allocator extended version of the make facility. We deem such complexity unnecessary with the advent of the CTAD feature.

Allocator aware types are assumed to be move enabled

`basic_optional` uses explicit allocator construction except in cases where invoking a direct `value_type` operation allows for move operations to remain noexcept. This is the case in move constructor for all allocators, and in move assignment for allocators that have `propagate_on_container_move_assignment==true`. If the `value_type` is allocator aware, but does not support move semantics (i.e. moves deteriorate to copies), it is possible that the allocator of the `value_type` object will get out of sync with the allocator of the `basic_optional`. We do not expect such types to exist.

Feedback items for LEWG:

Should assignments use `is_constructible` trait ?

It's not construct from U we need, it's construct from U using the allocator. Do we need a new trait ?

Do we need an “allocator_aware” concept?

We might consider an “allocator aware” (AA) concept that requires a `get_allocator()` method that returns something convertible to `std::pmr::polymorphic_allocator<>`. If we used this concept instead of the `std::uses_allocator` trait, the allocator would require zero extra space in most (but not all) cases. (The exception would be a type that stores its allocator outside of its footprint and whose footprint is smaller than `polymorphic_allocator<>`.)

Proposed wording

Add new paragraphs after 20.6.1/p1 [optional.general]

[2] `basic_optional` is a class template such that for a type `T` and allocator `Alloc`, an optional object of type `basic_optional<T&, Allocator>` will construct the contained object, if any, with `uses_allocator` construction.

Modify 20.6.2 Header synopsis1 [optional.syn]

```
namespace std {
    // [optional.optional], class template optional
    template<class T>
    class optional;
    // [optional.basic optional], class template basic optional
    template<class T, class Allocator = allocator<remove_cv_t<T>>>
    class basic_optional;
}
namespace pmr {
    template<class T>
    using optional = std::basic_optional<T, polymorphic_allocator<T>>;
}
...
```

Modify 20.6.3 Class template optional [optional.optional]

```
// [optional.ctor], constructors
...
template<class U, class Allocator = allocator<U>>
    explicit(see below) optional(const basic_optional<U, Allocator>&);
template<class U, class Allocator = allocator<U>>
    explicit(see below) optional(basic_optional<U, Allocator>&&);
...
// [optional.assign], assignment
...
template<class U, class Allocator = allocator<U>> optional&
    operator=(const basic_optional<U, Allocator>&);
template<class U, class Allocator = allocator<U>> optional&
    operator=(basic_optional<U, Allocator>&&) noexcept(see below);
...
template<class T>
```

```
optional(T) -> optional<T>;
template<class T, class Allocator>
optional(const basic_optional<T, Allocator>&&) -> optional<T>;
template<class T, class Allocator>
optional(basic_optional<T, Allocator>&&&) -> optional<T>;
```

Modify 20.6.3.1 Constructors [optional.ctor]

...

```
template<class U, class Allocator> explicit(see below) optional(const basic_optional<U, Allocator>& rhs);
```

Effects: If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type T with the expression *rhs.

Ensures: bool(rhs) == bool(*this).

Throws: Any exception thrown by the selected constructor of T.

Remarks:

The expression inside explicit is equivalent to:

```
!is_convertible_v<const U&, T>
```

Mandates:

- is constructible v<T, const U&> is true.
- is same<T, U> is false.
- is constructible v<T, optional<U>&> is false.
- is constructible v<T, basic_optional<U, Allocator>&> is false.
- is constructible v<T, optional<U>&&> is false.
- is constructible v<T, basic_optional<U, Allocator>&&> is false.
- is constructible v<T, const optional<U>&> is false.
- is constructible v<T, const basic_optional<U, Allocator>&> is false.
- is constructible v<T, const optional<U>&&> is false.
- is constructible v<T, const basic_optional<U, Allocator>&&> is false.
- is convertible v<optional<U>&, T> is false.
- is convertible v<basic_optional<U, Allocator>&, T> is false.
- is convertible v<optional<U>&&, T> is false.
- is convertible v<basic_optional<U, Allocator>&&, T> is false.
- is convertible v<const optional<U>&, T> is false.
- is convertible v<const basic_optional<U, Allocator>&, T> is false.
- is convertible v<const optional<U>&&, T> is false.
- is convertible v<const basic_optional<U, Allocator>&&, T> is false.

```
template<class U, class Allocator> explicit(see below) optional(basic_optional<U, Allocator>&& rhs);
```

Effects: If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type T with the expression std::move(*rhs).
std::move(*rhs) as the constructor argument. bool(rhs) is unchanged.

Ensures: bool(rhs) == bool(*this).

Throws: Any exception thrown by the selected constructor of T.

Remarks: The expression inside explicit is equivalent to:

```
!is_convertible_v<U, T>
```

Mandates:

- is constructible v<T, U&&> is true.
- is same<T, U> is false.
- is constructible v<T, optional<U>&> is false.
- is constructible v<T, basic_optional<U, Allocator>&> is false.
- is constructible v<T, optional<U>&&> is false.
- is constructible v<T, basic_optional<U, Allocator>&&> is false.
- is constructible v<T, const optional<U>&> is false.
- is constructible v<T, const basic_optional<U, Allocator>&> is false.
- is constructible v<T, const optional<U>&&> is false.
- is constructible v<T, const basic_optional<U, Allocator>&&> is false.
- is convertible v<optional<U>&, T> is false.
- is convertible v<basic_optional<U, Allocator>&, T> is false.
- is convertible v<optional<U>&&, T> is false.
- is convertible v<basic_optional<U, Allocator>&&, T> is false.
- is convertible v<const optional<U>&, T> is false.
- is convertible v<const basic_optional<U, Allocator>&, T> is false.
- is convertible v<const optional<U>&&, T> is false.
- is convertible v<const basic_optional<U, Allocator>&&, T> is false.

Modify 20.6.3.3 Assignment [optional.assign]

...

```
template<class U, class Allocator> optional& operator=(const basic_optional<U, Allocator>& rhs);
```

Effects: See Table

Table — optional operator=(const basic_optional<U, Allocator>&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns *rhs to the contained value	initializes the contained value as if direct-non-list-initializing an object of type T with *rhs.
rhs does not contain a value	destroys the contained value by calling val->T::~~T().	no effect

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to T's constructor, the state of `*rhs.val` is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of T's assignment.

Mandates:

- is same<T,U> is false.
- is constructible v<T, const U&> is true.
- is assignable<T&, const U&> is true.
- is constructible v<T, optional<U>&> is false.
- is constructible v<T, basic optional<U, Allocator>&> is false.
- is constructible v<T, optional<U>&&> is false.
- is constructible v<T, basic optional<U, Allocator>&&> is false.
- is constructible v<T, const optional<U>&> is false.
- is constructible v<T, const basic optional<U, Allocator>&> is false.
- is constructible v<T, const optional<U>&&> is false.
- is constructible v<T, const basic optional<U, Allocator>&&> is false.
- is convertible v<const U&, T> is true.
- is convertible v<optional<U>&, T> is false.
- is convertible v<basic optional<U, Allocator>&, T> is false.
- is convertible v<optional<U>&&, T> is false.
- is convertible v<basic optional<U, Allocator>&&, T> is false.
- is convertible v<const optional<U>&, T> is false.
- is convertible v<const basic optional<U, Allocator>&, T> is false.
- is convertible v<const optional<U>&&, T> is false.
- is convertible v<const basic optional<U, Allocator>&&, T> is false.
- is assignable v<T&, optional<U>&> is false.
- is assignable v<T&, basic optional<U, Allocator>&> is false.
- is assignable v<T&, optional<U>&&> is false.
- is assignable v<T&, basic optional<U, Allocator>&&> is false.
- is assignable v<T&, const optional<U>&> is false.
- is assignable v<T&, const basic optional<U, Allocator>&> is false.
- is assignable v<T&, const optional<U>&&> is false, and
- is assignable v<T&, const basic optional<U, Allocator>&&> is false

template<class U, class Allocator> optional& operator=(basic optional<U, Allocator>&& rhs);

Effects: See Table

Table — optional operator=(basic optional<U, Allocator>&&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns *rhs to the contained value	initializes the contained value as if direct-non-list-initializing an object of type T with <code>std::move(*rhs)</code> .
rhs does not contain a value	destroys the contained value by calling <code>val->T::~T()</code> .	no effect

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to T's constructor, the state of `*rhs.val` is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of T's assignment.

Mandates:

- is constructible v<T, const U&> is true.
- is assignable<T&, const U&> is true.
- is constructible v<T, optional<U>&> is false.
- is constructible v<T, basic optional<U, Allocator>&> is false.
- is constructible v<T, optional<U>&&> is false.
- is constructible v<T, basic optional<U, Allocator>&&> is false.
- is constructible v<T, const optional<U>&> is false.
- is constructible v<T, const basic optional<U, Allocator>&> is false.
- is constructible v<T, const optional<U>&&> is false.
- is constructible v<T, const basic optional<U, Allocator>&&> is false.
- is convertible v<const U&, T> is true.
- is convertible v<optional<U>&, T> is false.
- is convertible v<basic optional<U, Allocator>&, T> is false.
- is convertible v<optional<U>&&, T> is false.
- is convertible v<basic optional<U, Allocator>&&, T> is false.
- is convertible v<const optional<U>&, T> is false.
- is convertible v<const basic optional<U, Allocator>&, T> is false.
- is convertible v<const optional<U>&&, T> is false.
- is convertible v<const basic optional<U, Allocator>&&, T> is false.
- is assignable v<T&, optional<U>&> is false.
- is assignable v<T&, basic optional<U, Allocator>&> is false.
- is assignable v<T&, optional<U>&&> is false.
- is assignable v<T&, basic optional<U, Allocator>&&> is false.
- is assignable v<T&, const optional<U>&> is false.
- is assignable v<T&, const basic optional<U, Allocator>&> is false.
- is assignable v<T&, const optional<U>&&> is false, and
- is assignable v<T&, const basic optional<U, Allocator>&&> is false

Header <basic_optional> synopsis [basic_optional.sys]

```

namespace std {

    // [basic.optional.traits], allocator-related traits
    template<class T, class Allocator>
        struct uses_allocator<basic_optional<T, Allocator>;

    template<class T, class Allocator>
        constexpr bool uses_alloc = uses_allocator<T, Allocator>::value; // exposition only

    // [basic.optional.relops], relational operators
    template<class T, class AllocatorT, class U, class AllocatorU>
        bool operator==(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
    template<class T, class AllocatorT, class U, class AllocatorU>
        bool operator!=(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
    template<class T, class AllocatorT, class U, class AllocatorU>
        bool operator<(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
    template<class T, class AllocatorT, class U, class AllocatorU>
        bool operator>(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
    template<class T, class AllocatorT, class U, class AllocatorU>
        bool operator<=(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
    template<class T, class AllocatorT, class U, class AllocatorU>
        bool operator>=(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
    template<class T, class U, class Allocator>
        bool operator==(const basic_optional<T, Allocator>&, const optional<U>&);
    template<class T, class U, class Allocator>
        bool operator!=(const basic_optional<T, Allocator>&, const optional<U>&);
    template<class T, class U, class Allocator>
        bool operator<(const basic_optional<T, Allocator>&, const optional<U>&);
    template<class T, class U, class Allocator>
        bool operator>(const basic_optional<T, Allocator>&, const optional<U>&);
    template<class T, class U, class Allocator>
        bool operator<=(const basic_optional<T, Allocator>&, const optional<U>&);
    template<class T, class U, class Allocator>
        bool operator>=(const basic_optional<T, Allocator>&, const optional<U>&);
    template<class T, class U, class Allocator>
        bool operator==(const optional<T>&, const basic_optional<U, Allocator>&);
    template<class T, class U, class Allocator>
        bool operator!=(const optional<T>&, const basic_optional<U, Allocator>&);
    template<class T, class U, class Allocator>
        bool operator<(const optional<T>&, const basic_optional<U, Allocator>&);
    template<class T, class U, class Allocator>
        bool operator>(const optional<T>&, const basic_optional<U, Allocator>&);
    template<class T, class U, class Allocator>
        bool operator<=(const optional<T>&, const basic_optional<U, Allocator>&);
    template<class T, class U, class Allocator>
        bool operator>=(const optional<T>&, const basic_optional<U, Allocator>&);
    template<class T, class AllocatorT, three_way_comparable_with<T> U, class AllocatorU>
        compare_three_way_result_t<T,U>
            operator<=>(const basic_optional<T, AllocatorT>&, const basic_optional<U, AllocatorU>&);
    template<class T, three_way_comparable_with<T> U, class Allocator>
        compare_three_way_result_t<T,U>
            operator<=>(const basic_optional<T, Allocator>&, const optional<U>&);

    // [basic.optional.nullops], comparison with nullptr
    template<class T, class Allocator> bool operator==(const basic_optional<T, Allocator>&, nullptr_t) noexcept;
    template<class T, class Allocator>
        strong_ordering operator<=>(const basic_optional<T, Allocator>&, nullptr_t) noexcept;

    // [basic.optional.comp.with.tl], comparison with T
    template<class T, class U, class Allocator> bool operator==(const basic_optional<T, Allocator>&, const U&);
    template<class T, class U, class Allocator> bool operator==(const T&, const basic_optional<U, Allocator>&);
    template<class T, class U, class Allocator> bool operator!=(const basic_optional<T, Allocator>&, const U&);
    template<class T, class U, class Allocator> bool operator!=(const T&, const basic_optional<U, Allocator>&);
    template<class T, class U, class Allocator> bool operator<(const basic_optional<T, Allocator>&, const U&);
    template<class T, class U, class Allocator> bool operator<(const T&, const basic_optional<U, Allocator>&);
    template<class T, class U, class Allocator> bool operator>(const basic_optional<T, Allocator>&, const U&);
    template<class T, class U, class Allocator> bool operator>(const T&, const basic_optional<U, Allocator>&);
    template<class T, class U, class Allocator> bool operator<=(const basic_optional<T, Allocator>&, const U&);
    template<class T, class U, class Allocator> bool operator<=(const T&, const basic_optional<U, Allocator>&);
    template<class T, class U, class Allocator> bool operator>=(const basic_optional<T, Allocator>&, const U&);
    template<class T, class U, class Allocator> bool operator>=(const T&, const basic_optional<U, Allocator>&);
    template<class T, three_way_comparable_with<T> U, class Allocator>
        compare_three_way_result_t<T,U>
            operator<=>(const basic_optional<T, Allocator>&, const U&);

    // [basic.optional.special], specialized algorithms
    template<class T, class Allocator>
        void swap(basic_optional<T, Allocator>&, basic_optional<T, Allocator>&);

    template<class T>
        constexpr basic_optional<see below> make_basic_optional(T&&);
    template<class T>
        constexpr basic_optional<T> make_basic_optional();
    template<class T, class Arg0, class... Args>
        constexpr basic_optional<T> make_basic_optional(Arg0 arg0&&, Args&&... args);
    template<class T, class U, class... Args>
        constexpr basic_optional<T> make_basic_optional(initializer_list<U> il, Args&&... args);

    template<class T, class Allocator>
        constexpr basic_optional<see below> make_basic_optional(allocator_arg_t, const Allocator& a, T&&);
    template<class T, class Allocator, class... Args>
        constexpr basic_optional<T, Allocator> make_basic_optional(allocator_arg_t, const Allocator& a, Args&&... args);

```

```

    template<class T, class Allocator, class U, class... Args>
        constexpr basic optional<T, Allocator> make basic optional(allocator arg t, const Allocator& a,
initializer list<U> il, Args&&... args);

    // [basic.optional.hash], hash support
    template<class T, class Allocator> struct hash<optional<T, class Allocator>>;
}

namespace pmr {
    // [basic.pmroptional.specalg], specialized algorithms
    template<class T>
        constexpr optional<see below> make optional(T&&);
    template<class T>
        constexpr optional<T> make optional();
    template<class T, class Arg0, class... Args>
        constexpr optional<T> make optional(Arg0 arg0&&, Args&&... args);
    template<class T, class U, class... Args>
        constexpr optional<T> make optional(initializer list<U> il, Args&&... args);

    template<class T>
        constexpr optional<see below> make optional(allocator arg t, const polymorphic allocator<>& a, T&&);
    template<class T, class... Args>
        constexpr optional<T> make optional(allocator arg t, const polymorphic allocator<>& a, Args&&... args);
    template<class T, class U, class... Args>
        constexpr optional<T> make optional(allocator arg t, const polymorphic allocator<>& a, initializer list<U> il,
Args&&... args);
}

```

Class template basic optional [basic.optional]

```

namespace std {
    template<class T, class Allocator>
        class basic optional see below{
    public:
        using value type = T;
        using allocator type = Allocator<>; // not always defined, see below

        // [basic.optional.ctor], constructors
        basic optional() noexcept;
        basic optional(nullopt t) noexcept;
        basic optional(const basic optional&);
        basic optional(optional&&) noexcept(see below);
        basic optional(const optional&);
        basic optional(optional&&);
        template<class... Args>
            explicit basic optional(in place t, Args&&...);
        template<class U, class... Args>
            explicit basic optional(in place t, initializer list<U>, Args&&...);
        template<class U = T>
            explicit(see below) basic optional(U&&);
        template<class U>
            explicit(see below) basic optional(const basic optional<U, Allocator>&);
        template<class U>
            explicit(see below) basic optional(optional<U, Allocator>&&);
        template<class U>
            explicit(see below) basic optional(const optional<U>&);
        template<class U>
            explicit(see below) basic optional(optional<U>&&);

        // allocator-extended constructors
        optional(allocator arg t, const Allocator<>&) noexcept; // not always provided, see below
        optional(allocator arg t, const Allocator<>&, nullopt t) noexcept; // not always provided, see below
        optional(allocator arg t, const Allocator<>&, const basic optional&); // not always provided, see below
        optional(allocator arg t, const Allocator<>&, basic optional&) noexcept(see below); // not always provided, see
below
        optional(allocator arg t, const Allocator<>&, const optional&); // not always provided, see below
        optional(allocator arg t, const Allocator<>&, optional&&); // not always provided, see below
        template<class... Args>
            explicit basic optional(allocator arg t, const Allocator<>&, in place t, Args&&...); // not always provided,
see below
        template<class U, class... Args>
            explicit basic optional(allocator arg t, const Allocator<>&, in place t, initializer list<U>, Args&&...); //
not always provided, see below
        template<class U = T>
            explicit(see below) basic optional(allocator arg t, const Allocator<>&, U&&); // not always provided, see
below
        template<class U>
            explicit(see below) basic optional(allocator arg t, const Allocator<>&, const basic optional<U, Allocator>&);
// not always provided, see below
        template<class U>
            explicit(see below) basic optional(allocator arg t, const Allocator<>&, optional<U, Allocator>&&); // not
always provided, see below
        template<class U>
            explicit(see below) basic optional(allocator arg t, const Allocator<>&, const optional<U>&); // not always
provided, see below
        template<class U>
            explicit(see below) basic optional(allocator arg t, const Allocator<>&, optional<U>&&); // not always
provided, see below

        // [basic.optional.dtor], destructor
        ~basic optional();

        // [basic.optional.assian], assignment
        basic optional& operator=(nullopt t) noexcept;
        basic optional& operator=(const basic optional&);
        basic optional& operator=(optional&&) noexcept(see below);

```

```

template<class U = T> basic optional& operator=(U&&);
template<class U> basic optional& operator=(const basic optional<U, Allocator>&);
template<class U> basic optional& operator=(basic optional<U, Allocator>&&);
template<class U, class AllocatorU> basic optional& operator=(const basic optional<U, AllocatorU>&);
template<class U, class AllocatorU> basic optional& operator=(basic optional<U, AllocatorU>&&);
template<class U> basic optional& operator=(const optional<U>&);
template<class U> basic optional& operator=(optional<U>&&);
template<class... Args>> T& emplace(Args&&...);
template<class U, class... Args>> T& emplace(initializer_list<U>, Args&&...);

// [basic.optional.swap], swap
void swap(basic optional&) noexcept(see below);

// [basic.optional.observe], observers
const T* operator->() const;
T* operator->();
const T& operator*() const&;
T& operator*() &;
T&& operator*() &&;
const T&& operator*() const&&;
explicit operator bool() const noexcept;
bool has_value() const noexcept;
const T& value() const&;
T& value() &;
T&& value() &&;
const T&& value() const&&;
template<class U> T value or(U&&) const&;
template<class U> T value or(U&&) &&;
template<class U> T value or(allocator arg t, const Allocator<>&, U&&) const&; // not always provided, see below
template<class U> T value or(allocator arg t, const Allocator<>&, U&&) &&; // not always provided, see below
allocator_type get_allocator() const noexcept;

// [basic.optional.mod], modifiers
void reset() noexcept;

private:
    T* val; // exposition only
    const Allocator<> alloc; // exposition only; not always provided, see below
};

template<class T>
basic optional(T)-> basic optional<T>;
template<class T>
basic optional(const optional<T>&)-> basic optional<T>;
template<class T>
basic optional(optional<T>&&)-> basic optional<T>;

```

};

Any instance of `basic optional<T, Allocator>` at any given time either contains a value or does not contain a value. When an instance of `basic optional<T, Allocator>` contains a value, it means that an object of type `T` is allocated within the storage of the `basic optional<T, Allocator>` object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained value. The contained value shall be allocated in a region of the `basic optional<T, Allocator>` storage suitably aligned for the type `T` and, if `uses-alloc` is true, constructed by `uses-allocator` construction with allocator `alloc`. When an object of type `basic optional<T, Allocator>` is contextually converted to `bool`, the conversion returns `true` if the object contains a value; otherwise the conversion returns `false`.

Member `val` is provided for exposition only. When an `basic optional<T, Allocator>` object contains a value, `val` points to the contained value.

If and only if `uses-alloc` is true, the following applies:

- `allocator_type` will be defined.
- exposition only member `alloc` will be provided and will be used to create the contained value, if any, and
- if `T` provides a `get_allocator()` member function whose return type is convertible to `Allocator<>`, the implementation is allowed to assume that the allocator used for constructing the element is the same allocator as returned by `T::get_allocator()`. [Note: In such an implementation, `pmr::optional<pmr::string>` will be zero-overhead compared to `optional<pmr::string>`. -- end note]

`T` shall be a type other than `cv in_place_t`, `cv allocator_arg_t`, or `cv nullptr_t` that meets the `Cpp17Destructible` requirements (Table 30).

Constructors [basic.optional.ctor]

```

basic optional() noexcept;
basic optional(nullptr_t) noexcept;
Effects: If uses-alloc is true, alloc is default initialized
Ensures: *this does not contain a value.
Remarks: No contained value is initialized.

```

```

basic optional(const basic optional& rhs);
Effects: If uses-alloc is true, alloc is initialized from
allocator_traits<Allocator>::select_on_container_copy_construction(rhs.get_allocator()). If rhs contains a value, constructs the contained
value with uses-allocator construction ([allocator.uses.construction]) with alloc and the expression *rhs.
Ensures: bool(rhs) == bool(*this).
Throws: Any exception thrown by the selected constructor of T.
Mandates: is_copy_constructible v<T> is true.

```

```

basic optional(basic optional&& rhs) noexcept(see below);
Effects: If uses-alloc is true, alloc is initialized with rhs.alloc. If rhs contains a value, initialized the contained value the expression std::
move(*rhs). bool(rhs) is unchanged.
Ensures: bool(rhs) == bool(*this).
Throws: Any exception thrown by the selected constructor of T.

```

Remarks: The expression inside `noexcept` is equivalent to `is_nothrow_move_constructible v<T>`. If type is not move enabled, the allocator of the resulting value type object may differ from `alloc`.
Mandates: `is_move_constructible v<T>` is true.

`basic_optional(const optional&);`

Effects: If `uses-alloc` is true, `alloc` is default initialized. If `rhs` contains a value, initializes the contained value with the expression `*rhs`.
Ensures: `bool(rhs) == bool(*this)`.
Throws: Any exception thrown by the selected constructor of `T`.
Mandates: `is_copy_constructible v<T>` is true.

`basic_optional(optional&& rhs);`

Effects: If `uses-alloc` is true, `alloc` is default initialized. If `rhs` contains a value, constructs the contained value with `uses-allocator construction` (`(allocator.uses.construction)`) with `alloc` and the expression `std::move(*rhs)`. `bool(rhs)` is unchanged.
Ensures: `bool(rhs) == bool(*this)`.
Throws: Any exception thrown by the selected constructor of `T`.
Mandates: `is_move_constructible v<T>` is true.

`template<class... Args> explicit basic_optional(in_place_t, Args&&... args);`

Effects: If `uses-alloc` is true, `alloc` is default initialized. Constructs the contained value with `uses-allocator construction` (`(allocator.uses.construction)`) with `alloc` and `std::forward<Args>(args)....`.
Ensures: `*this` contains a value.
Throws: Any exception thrown by the selected constructor of `T`.
Mandates: `is_copy_constructible v<T>` is true.

`template<class... Args> explicit basic_optional(in_place_t, initializer_list<U> il, Args&&... args);`

Effects: If `uses-alloc` is true, `alloc` is default initialized. Constructs the contained value with `uses-allocator construction` (`(allocator.uses.construction)`) with `alloc`, `il`, and `std::forward<Args>(args)....`.
Ensures: `*this` contains a value.
Throws: Any exception thrown by the selected constructor of `T`.
Mandates: `is_constructible v<T, initializer_list<U>&, Args&&...>` is true.

`template<class U = T> explicit(see below) basic_optional(U&& rhs);`

Effects: If `uses-alloc` is true, `alloc` is default initialized. Constructs the contained value with `uses-allocator construction` (`(allocator.uses.construction)`) with `alloc` and `std::forward<U>(rhs)`.
Ensures: `*this` contains a value.
Throws: Any exception thrown by the selected constructor of `T`.
Remarks: The expression inside `explicit` is equivalent to:
`!is_convertible v<U, T>`
Mandates:
- `is_constructible v<T, U&&>` is true.
- `is_same v<remove_cvref t<U>, in_place_t>` is false, and
- `is_same v<remove_cvref t<U>, optional>` is false.

`template<class U> explicit(see below) basic_optional(const basic_optional<U, Allocator>& rhs);`

Effects: If `uses-alloc` is true, `alloc` is default initialized. If `rhs` contains a value, constructs the contained value with `uses-allocator construction` (`(allocator.uses.construction)`) with `alloc` and `*rhs`.
Ensures: `bool(rhs) == bool(*this)`.
Throws: Any exception thrown by the selected constructor of `T`.
Remarks: The expression inside `explicit` is equivalent to:
`!is_convertible v<const U&, T>`
Mandates:
- `is_constructible v<T, const U&>` is true.
- `is_constructible v<T, optional<U>&>` is false.
- `is_constructible v<T, basic_optional<U, Allocator>&>` is false.
- `is_constructible v<T, optional<U>&&>` is false.
- `is_constructible v<T, basic_optional<U, Allocator>&&>` is false.
- `is_constructible v<T, const optional<U>&>` is false.
- `is_constructible v<T, const basic_optional<U, Allocator>&>` is false.
- `is_constructible v<T, const optional<U>&&>` is false.
- `is_constructible v<T, const basic_optional<U, Allocator>&&>` is false.
- `is_convertible v<optional<U>&, T>` is false.
- `is_convertible v<optional<U>&, T>` is false.
- `is_convertible v<optional<U>&&, T>` is false.
- `is_convertible v<optional<U>&&, T>` is false.
- `is_convertible v<const optional<U>&, T>` is false.
- `is_convertible v<const basic_optional<U, Allocator>&, T>` is false.
- `is_convertible v<const optional<U>&&, T>` is false, and
- `is_convertible v<const basic_optional<U, Allocator>&&, T>` is false.

`template<class U> explicit(see below) basic_optional(basic_optional<U, Allocator>&& rhs);`

Effects: If `uses-alloc` is true, `alloc` is default initialized. If `rhs` contains a value, constructs the contained value with `uses-allocator construction` (`(allocator.uses.construction)`) with `alloc` and `std::move(*rhs)`. `bool(rhs)` is unchanged.
Ensures: `bool(rhs) == bool(*this)`.
Throws: Any exception thrown by the selected constructor of `T`.
Remarks: The expression inside `explicit` is equivalent to:
`!is_convertible v<U, T>` *Mandates:*
- `is_constructible v<T, U&&>` is true.
- `is_constructible v<T, optional<U>&>` is false.
- `is_constructible v<T, basic_optional<U, Allocator>&>` is false.
- `is_constructible v<T, optional<U>&&>` is false.
- `is_constructible v<T, basic_optional<U, Allocator>&&>` is false.

- is constructible `v<T, const optional<U>&>` is false.
- is constructible `v<T, const basic optional<U, Allocator>&>` is false.
- is constructible `v<T, const optional<U>&&>` is false.
- is constructible `v<T, const basic optional<U, Allocator>&&>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&&, T>` is false.
- is convertible `v<optional<U>&&, T>` is false.
- is convertible `v<optional<U>&&, T>` is false.
- is convertible `v<const optional<U>&, T>` is false.
- is convertible `v<const basic optional<U, Allocator>&, T>` is false.
- is convertible `v<const optional<U>&&, T>` is false. and
- is convertible `v<const basic optional<U, Allocator>&&, T>` is false

`template<class U> explicit(see below) basic optional(const optional<U>& rhs);`

Effects: If `uses-alloc` is true, `alloc` is default initialized. If `rhs` contains a value, constructs the contained value with uses-allocator construction (`(allocator.uses.construction)`) with `alloc` and `*rhs`.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: The expression inside `explicit` is equivalent to:

`!is convertible v<const U&, T>`

Mandates:

- is constructible `v<T, const U&>` is true.
- is constructible `v<T, optional<U>&>` is false.
- is constructible `v<T, basic optional<U, Allocator>&>` is false.
- is constructible `v<T, optional<U>&&>` is false.
- is constructible `v<T, basic optional<U, Allocator>&&>` is false.
- is constructible `v<T, const optional<U>&>` is false.
- is constructible `v<T, const basic optional<U, Allocator>&>` is false.
- is constructible `v<T, const optional<U>&&>` is false.
- is constructible `v<T, const basic optional<U, Allocator>&&>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&&, T>` is false.
- is convertible `v<const optional<U>&, T>` is false.
- is convertible `v<const basic optional<U, Allocator>&, T>` is false.
- is convertible `v<const optional<U>&&, T>` is false. and
- is convertible `v<const basic optional<U, Allocator>&&, T>` is false

`template<class U> explicit(see below) basic optional(optional<U>&& rhs);`

Effects: If `uses-alloc` is true, `alloc` is default initialized. If `rhs` contains a value, constructs the contained value with uses-allocator construction (`(allocator.uses.construction)`) with `alloc` and `std::move(*rhs)`. `bool(rhs)` is unchanged.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks:

The expression inside `explicit` is equivalent to:

`!is convertible v<U, T>`

Mandates:

- is constructible `v<T, U&&>` is true.
- is constructible `v<T, optional<U>&>` is false.
- is constructible `v<T, basic optional<U, Allocator>&>` is false.
- is constructible `v<T, optional<U>&&>` is false.
- is constructible `v<T, basic optional<U, Allocator>&&>` is false.
- is constructible `v<T, const optional<U>&>` is false.
- is constructible `v<T, const basic optional<U, Allocator>&>` is false.
- is constructible `v<T, const optional<U>&&>` is false.
- is constructible `v<T, const basic optional<U, Allocator>&&>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&&, T>` is false.
- is convertible `v<optional<U>&&, T>` is false.
- is convertible `v<const optional<U>&, T>` is false.
- is convertible `v<const basic optional<U, Allocator>&, T>` is false.
- is convertible `v<const optional<U>&&, T>` is false. and
- is convertible `v<const basic optional<U, Allocator>&&, T>` is false

`optional(allocator arg t, const Allocator<>&) noexcept;`

`optional(allocator arg t, const Allocator<>&, nullptr t) noexcept;`

`optional(allocator arg t, const Allocator<>&, const basic optional&);`

`optional(allocator arg t, const Allocator<>&, basic optional&&) noexcept(see below);`

`optional(allocator arg t, const Allocator<>&, const optional&);`

`optional(allocator arg t, const Allocator<>&, optional&&);`

`template<class... Args> explicit basic optional(allocator arg t, const Allocator<>&, in place t, Args&&...);`

`template<class U, class... Args> explicit basic optional(allocator arg t, const Allocator<>&, in place t, initializer list<U>, Args&&...);`

`template<class U = T> explicit(see below) basic optional(allocator arg t, const Allocator<>&, U&&);`

`template<class U> explicit(see below) basic optional(allocator arg t, const Allocator<>&, const basic optional<U, Allocator>&);`

`template<class U>`

```
explicit(see below) basic_optional(allocator arg t, const Allocator<>&, basic_optional<UAllocator>&&);
template<class U>
explicit(see below) basic_optional(allocator arg t, const Allocator<>&, const optional<U>&);
template<class U>
explicit(see below) basic_optional(allocator arg t, const Allocator<>&, optional<U>&&);
Constraints: uses_allocator<T, Allocator<>> is true;
Effects: Equivalent to the preceding constructors except that alloc is initialized with a.
```

Destructor [basic.optional.dtor]

```
-basic_optional().;
Effects: If is_trivially_destructible v<T> != true and *this contains a value, calls val->T::~~T().
Remarks: If is_trivially_destructible v<T> is true, then this destructor is trivial.
```

Assignment [basic.optional.assign]

```
basic_optional<T, Allocator>& operator=(nullopt t) noexcept;
Effects: If *this contains a value, calls val->T::~~T() to destroy the contained value; otherwise no effect.
Remarks: *this
Ensures: *this does not contain a value.
```

```
basic_optional<T, Allocator>& operator=(const basic_optional& rhs);
Effects: See Table
```

Table — basic_optional operator=(const basic_optional& rhs) effects

	*this contains a value	*this does not contain a value
rhs contains a value	If allocator_traits<Allocator>::propagate_on_container_copy_assignment and this->get_allocator() != rhs->get_allocator() is true: destroys the contained value by calling val->T::~~T(). propagates the allocator, and initializes the contained value with *rhs by uses-allocator construction with allocator alloc otherwise: assigns *rhs to the contained value	If allocator_traits<Allocator>::propagate_on_container_copy_assignment and this->get_allocator() != rhs->get_allocator() is true, propagates the allocator. Initializes the contained value with *rhs. If uses_alloc is true, is done by uses-allocator construction with allocator alloc
rhs does not contain a value	If allocator_traits<Allocator>::propagate_on_container_copy_assignment and this->get_allocator() != rhs->get_allocator() is true, propagates the allocator. Destroys the contained value by calling val->T::~~T().	If allocator_traits<Allocator>::propagate_on_container_copy_assignment and this->get_allocator() != rhs->get_allocator() is true, propagates the allocator.

Returns: *this.
Ensures: bool(rhs) == bool(*this).
Remarks: If any exception is thrown, the result of the expression bool(*this) remains unchanged. If an exception is thrown during the call to T's copy constructor, no effect. If an exception is thrown during the call to T's copy assignment, the state of of its contained value is as defined by the exception safety guarantee of is determined by the exception safety guarantee of T's copy assignment. This operator shall be defined as deleted unless is_copy_constructible v<T> is true and is_copy_assignable v<T> is true.

```
basic_optional<T, Allocator>& operator=(basic_optional&& rhs) noexcept(see below);
Effects: See Table
```

Table — basic_optional operator=(basic_optional&& rhs) effects

	*this contains a value	*this does not contain a value
rhs contains a value	If allocator_traits<Allocator>::propagate_on_container_move_assignment is true, propagates the allocator. Assigns std::move(*rhs) to the contained value	If allocator_traits<Allocator>::propagate_on_container_move_assignment - propagates the allocator - Initializes the contained value with std::move(*rhs). Otherwise: Initializes the contained value with std::move(*rhs). If uses_allocator is true, initialization is done by uses-allocator construction with allocator alloc
rhs does not contain a value	If allocator_traits<Allocator>::propagate_on_container_move_assignment, propagates the allocator. Destroys the contained value by calling val->T::~~T().	If allocator_traits<Allocator>::propagate_on_container_move_assignment, propagates the allocator.

Returns: *this.
Ensures: bool(rhs) == bool(*this).
Remarks: The expression inside noexcept is equivalent to:
is_nothrow_move_assignable v<T> && is_nothrow_move_constructible v<T>
If any exception is thrown, the result of the expression bool(*this) remains unchanged. If an exception is thrown during the call to T's move constructor, the state of *rhs.val is determined by the exception safety guarantee of T's move constructor. If an exception is thrown during the call to T's move assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's move assignment.
Mandates: is_move_constructible v<T> is true and is_move_assignable v<T> is true. If type is not move enabled, the allocator of the resulting value type object may differ from alloc

```
template<class U = T> basic_optional& operator=(U&& v).;
```

Effects: If `*this` contains a value, assigns `std::forward<U>(v)` to the contained value; otherwise initializes the contained value with `std::forward<U>(v)`. If `uses-alloc` is true, initialization is done by `uses-allocator` construction with allocator `alloc`.

Returns: `*this`.

Ensures: `*this` contains a value.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `v` is determined by the exception safety guarantee of `T`'s assignment. Any allocator propagation resulting from assignment of `std::forward<U>(v)` to the contained value will not be reflected in the basic optional's allocator.

Mandates:

- is same `v<remove_cvref_t<U>, alloc-optional>` is false.
- conjunction `v<is_scalar<T>, is_same<T, decay_t<U>>>` is false.
- is constructible `v<T,U>` is true, and
- is assignable `v<T&,U>` is true

template<class U> basic optional& operator=(const basic optional<U, Allocator>& rhs);

Effects: See Table

Table — basic optional operator=(const basic optional<U, Allocator>&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns <code>*rhs</code> to the contained value	initializes the contained value with <code>*rhs</code> . If <code>uses-alloc</code> is true, initialization is done by <code>uses-allocator</code> construction with allocator <code>alloc</code> .
rhs does not contain a value	destroys the contained value by calling <code>val->T::~T()</code> .	no effect

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, no effect. If an exception is thrown during the call to `T`'s assignment, the state of `*val` is determined by the exception safety guarantee of `T`'s assignment.

Mandates:

- is same `<T,U>` is false.
- is constructible `v<T, const U&>` is true.
- is assignable `<T&, const U&>` is true.
- is constructible `v<T, optional<U>&>` is false.
- is constructible `v<T, basic optional<U, Allocator>&>` is false.
- is constructible `v<T, optional<U>&&>` is false.
- is constructible `v<T, basic optional<U, Allocator>&&>` is false.
- is constructible `v<T, const optional<U>&>` is false.
- is constructible `v<T, const basic optional<U, Allocator>&>` is false.
- is constructible `v<T, const optional<U>&&>` is false.
- is constructible `v<T, const basic optional<U, Allocator>&&>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&&, T>` is false.
- is convertible `v<optional<U>&&, T>` is false.
- is convertible `v<const optional<U>&, T>` is false.
- is convertible `v<const basic optional<U, Allocator>&, T>` is false.
- is convertible `v<const optional<U>&&, T>` is false.
- is convertible `v<const basic optional<U, Allocator>&&, T>` is false.
- is assignable `v<T&, optional<U>&>` is false.
- is assignable `v<T&, basic optional<U, Allocator>&>` is false.
- is assignable `v<T&, optional<U>&&>` is false.
- is assignable `v<T&, basic optional<U, Allocator>&&>` is false.
- is assignable `v<T&, const optional<U>&>` is false.
- is assignable `v<T&, const basic optional<U, Allocator>&>` is false.
- is assignable `v<T&, const optional<U>&&>` is false, and
- is assignable `v<T&, const basic optional<U, Allocator>&&>` is false

template<class U> basic optional& operator=(basic optional<U, Allocator>&& rhs);

Effects: See Table

Table — basic optional operator=(basic optional<U, Allocator>&&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns <code>*rhs</code> to the contained value	initializes the contained value with <code>*rhs</code> . If <code>uses-alloc</code> is true, initialization is done by <code>uses-allocator</code> construction with allocator <code>alloc</code> .
rhs does not contain a value	destroys the contained value by calling <code>val->T::~T()</code> .	no effect

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s move assignment.

Mandates:

- is same `<T,U>` is false.
- is constructible `v<T, const U&>` is true.
- is assignable `<T&, const U&>` is true.
- is constructible `v<T, optional<U>&>` is false.

- is constructible $v < T, \text{basic optional} < U, \text{Allocator} > \& >$ is false.
- is constructible $v < T, \text{optional} < U > \& \& >$ is false.
- is constructible $v < T, \text{basic optional} < U, \text{Allocator} > \& \& >$ is false.
- is constructible $v < T, \text{const optional} < U > \& >$ is false.
- is constructible $v < T, \text{const basic optional} < U, \text{Allocator} > \& >$ is false.
- is constructible $v < T, \text{const optional} < U > \& \& >$ is false.
- is constructible $v < T, \text{const basic optional} < U, \text{Allocator} > \& \& >$ is false.
- is convertible $v < \text{const } U \&, T >$ is true.
- is convertible $v < \text{optional} < U > \&, T >$ is false.
- is convertible $v < \text{optional} < U > \&, T >$ is false.
- is convertible $v < \text{optional} < U > \& \&, T >$ is false.
- is convertible $v < \text{optional} < U > \& \&, T >$ is false.
- is convertible $v < \text{const optional} < U > \&, T >$ is false.
- is convertible $v < \text{const basic optional} < U, \text{Allocator} > \&, T >$ is false.
- is convertible $v < \text{const optional} < U > \& \&, T >$ is false.
- is convertible $v < \text{const basic optional} < U, \text{Allocator} > \& \&, T >$ is false.
- is assignable $v < T \&, \text{optional} < U > \& >$ is false.
- is assignable $v < T \&, \text{basic optional} < U, \text{Allocator} > \& >$ is false.
- is assignable $v < T \&, \text{optional} < U > \& \& >$ is false.
- is assignable $v < T \&, \text{basic optional} < U, \text{Allocator} > \& \& >$ is false.
- is assignable $v < T \&, \text{const optional} < U > \& >$ is false.
- is assignable $v < T \&, \text{const basic optional} < U, \text{Allocator} > \& >$ is false.
- is assignable $v < T \&, \text{const optional} < U > \& \& >$ is false. and
- is assignable $v < T \&, \text{const basic optional} < U, \text{Allocator} > \& \& >$ is false

```
template<class U, class Allocator> basic_optional& operator=(const basic_optional<U, Allocator>& rhs);
```

Effects: See Table

Table — basic_optional operator=(const basic_optional<U, Allocator>&) effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	assigns *rhs to the contained value	initializes the contained value with *rhs. If uses-alloc is true, initialization is done by uses-allocator construction with allocator alloc
<u>rhs does not contain a value</u>	destroys the contained value by calling <code>val->T::~T()</code> .	no effect

Returns: *this.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to T's constructor, no effect. If an exception is thrown during the call to T's assignment, the state of *val is determined by the exception safety guarantee of T's assignment.

Mandates:

- is same<T, U> or is same<Allocator, AllocatorU> is false.
- is constructible $v < T, \text{const } U \& >$ is true.
- is assignable $v < T \&, \text{const } U \& >$ is true.
- is constructible $v < T, \text{optional} < U > \& >$ is false.
- is constructible $v < T, \text{basic optional} < U, \text{Allocator} > \& >$ is false.
- is constructible $v < T, \text{optional} < U > \& \& >$ is false.
- is constructible $v < T, \text{basic optional} < U, \text{Allocator} > \& \& >$ is false.
- is constructible $v < T, \text{const optional} < U > \& >$ is false.
- is constructible $v < T, \text{const basic optional} < U, \text{Allocator} > \& >$ is false.
- is constructible $v < T, \text{const optional} < U > \& \& >$ is false.
- is constructible $v < T, \text{const basic optional} < U, \text{Allocator} > \& \& >$ is false.
- is convertible $v < \text{optional} < U > \&, T >$ is false.
- is convertible $v < \text{optional} < U > \&, T >$ is false.
- is convertible $v < \text{optional} < U > \& \&, T >$ is false.
- is convertible $v < \text{optional} < U > \& \&, T >$ is false.
- is convertible $v < \text{const optional} < U > \&, T >$ is false.
- is convertible $v < \text{const basic optional} < U, \text{Allocator} > \&, T >$ is false.
- is convertible $v < \text{const optional} < U > \& \&, T >$ is false.
- is convertible $v < \text{const basic optional} < U, \text{Allocator} > \& \&, T >$ is false.
- is assignable $v < T \&, \text{optional} < U > \& >$ is false.
- is assignable $v < T \&, \text{basic optional} < U, \text{Allocator} > \& >$ is false.
- is assignable $v < T \&, \text{optional} < U > \& \& >$ is false.
- is assignable $v < T \&, \text{basic optional} < U, \text{Allocator} > \& \& >$ is false.
- is assignable $v < T \&, \text{const optional} < U > \& >$ is false.
- is assignable $v < T \&, \text{const basic optional} < U, \text{Allocator} > \& >$ is false.
- is assignable $v < T \&, \text{const optional} < U > \& \& >$ is false. and
- is assignable $v < T \&, \text{const basic optional} < U, \text{Allocator} > \& \& >$ is false

```
template<class U, class Allocator> basic_optional& operator=(basic_optional<U, Allocator>&& rhs);
```

Effects: See Table

Table — basic_optional operator=(basic_optional<U, Allocator>&&) effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	assigns <code>std::move(*rhs)</code> to the contained value	initializes the contained value with <code>std::move(*rhs)</code> . If uses-alloc is true, initialization is done by uses-allocator construction with allocator alloc.
<u>rhs does not contain a value</u>	destroys the contained value by calling <code>val->T::~T()</code> .	no effect

Returns: **this*.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to *T*'s move constructor, the state of **rhs.val* is determined by the exception safety guarantee of *T*'s move constructor. If an exception is thrown during the call to *T*'s move assignment, the state of **val* and **rhs.val* is determined by the exception safety guarantee of *T*'s move assignment.

Mandates:

- is same<*T*, *U*> or is same<Allocator, Allocator*U*> is false.
- is constructible *v*<*T*, const *U*&> is true.
- is assignable<*T*&, const *U*&> is true.
- is constructible *v*<*T*, optional<*U*>&> is false.
- is constructible *v*<*T*, basic optional<*U*, Allocator*U*>&> is false.
- is constructible *v*<*T*, optional<*U*>&&> is false.
- is constructible *v*<*T*, basic optional<*U*, Allocator*U*>&&> is false.
- is constructible *v*<*T*, const optional<*U*>&> is false.
- is constructible *v*<*T*, const basic optional<*U*, Allocator*U*>&> is false.
- is constructible *v*<*T*, const optional<*U*>&&> is false.
- is constructible *v*<*T*, const basic optional<*U*, Allocator*U*>&&> is false.
- is convertible *v*<const *U*&, *T*> is true.
- is convertible *v*<optional<*U*>&, *T*> is false.
- is convertible *v*<optional<*U*>&, *T*> is false.
- is convertible *v*<optional<*U*>&&, *T*> is false.
- is convertible *v*<optional<*U*>&&, *T*> is false.
- is convertible *v*<const optional<*U*>&, *T*> is false.
- is convertible *v*<const basic optional<*U*, Allocator*U*>&, *T*> is false.
- is convertible *v*<const optional<*U*>&&, *T*> is false.
- is convertible *v*<const basic optional<*U*, Allocator*U*>&&, *T*> is false.
- is assignable *v*<*T*&, optional<*U*>&> is false.
- is assignable *v*<*T*&, basic optional<*U*, Allocator*U*>&> is false.
- is assignable *v*<*T*&, optional<*U*>&&> is false.
- is assignable *v*<*T*&, basic optional<*U*, Allocator*U*>&&> is false.
- is assignable *v*<*T*&, const optional<*U*>&> is false.
- is assignable *v*<*T*&, const basic optional<*U*, Allocator*U*>&> is false.
- is assignable *v*<*T*&, const optional<*U*>&&> is false.
- is assignable *v*<*T*&, const basic optional<*U*, Allocator*U*>&&> is false.

template<class *U*> basic optional& operator=(const optional<*U*>& rhs);

Effects: The result of the expression `bool(rhs)` remains unchanged. See Table

Table — basic optional operator=(const optional<*U*>&) effects

	<i>*this</i> contains a value	<i>*this</i> does not contain a value
<i>rhs</i> contains a value	assigns <i>*rhs</i> to the contained value	initializes the contained value with <i>*rhs</i> . If <code>uses-alloc</code> is true, initialization is done by <code>uses-allocator</code> construction with allocator <code>alloc</code> .
<i>rhs</i> does not contain a value	destroys the contained value by calling <code>val->T::~T()</code> .	no effect

Returns: **this*.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to *T*'s constructor, no effect. If an exception is thrown during the call to *T*'s assignment, the state of **val* is determined by the exception safety guarantee of *T*'s assignment.

Any allocator propagation in assignment of **rhs* to the contained value will not be reflected in the basic optional's allocator.

Mandates:

- is constructible *v*<*T*, *U*> is true.
- is assignable<*T*&, *U*> is true.
- is constructible *v*<*T*, optional<*U*>&> is false.
- is constructible *v*<*T*, basic optional<*U*, Allocator>&> is false.
- is constructible *v*<*T*, optional<*U*>&&> is false.
- is constructible *v*<*T*, basic optional<*U*, Allocator>&&> is false.
- is constructible *v*<*T*, const optional<*U*>&> is false.
- is constructible *v*<*T*, const basic optional<*U*, Allocator>&> is false.
- is constructible *v*<*T*, const optional<*U*>&&> is false.
- is constructible *v*<*T*, const basic optional<*U*, Allocator>&&> is false.
- is convertible *v*<optional<*U*>&, *T*> is false.
- is convertible *v*<optional<*U*>&, *T*> is false.
- is convertible *v*<optional<*U*>&&, *T*> is false.
- is convertible *v*<optional<*U*>&&, *T*> is false.
- is convertible *v*<const optional<*U*>&, *T*> is false.
- is convertible *v*<const basic optional<*U*, Allocator>&, *T*> is false.
- is convertible *v*<const optional<*U*>&&, *T*> is false.
- is convertible *v*<const basic optional<*U*, Allocator>&&, *T*> is false.
- is assignable *v*<*T*&, optional<*U*>&> is false.
- is assignable *v*<*T*&, basic optional<*U*, Allocator>&> is false.
- is assignable *v*<*T*&, optional<*U*>&&> is false.
- is assignable *v*<*T*&, basic optional<*U*, Allocator>&&> is false.
- is assignable *v*<*T*&, const optional<*U*>&> is false.
- is assignable *v*<*T*&, const basic optional<*U*, Allocator>&> is false.
- is assignable *v*<*T*&, const optional<*U*>&&> is false.
- is assignable *v*<*T*&, const basic optional<*U*, Allocator>&&> is false.

```
template<class U> basic_optional& operator=(optional<U>&& rhs);
```

Effects: See Table

Table — `basic_optional operator=(optional<U>&&) effects`

	*this contains a value	*this does not contain a value
rhs contains a value	assigns <code>std::move(*rhs)</code> to the contained value	initializes the contained value with <code>std::move(*rhs)</code> . If <code>uses-alloc</code> is true, initialization is done by <code>uses-allocator</code> construction with <code>allocator alloc</code>
rhs does not contain a value	destroys the contained value by calling <code>val->T::~T()</code>	no effect

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s assignment.

Any allocator propagation in assignment of `std::move(*rhs)` to the contained value will not be reflected in the `basic_optional`'s allocator.

Mandates:

- is constructible `v<T, const U&>` is true.
- is assignable `v<T, const U&>` is true.
- is constructible `v<T, optional<U>&>` is false.
- is constructible `v<T, basic_optional<U, Allocator>&>` is false.
- is constructible `v<T, optional<U>&&>` is false.
- is constructible `v<T, basic_optional<U, Allocator>&&>` is false.
- is constructible `v<T, const optional<U>&>` is false.
- is constructible `v<T, const basic_optional<U, Allocator>&>` is false.
- is constructible `v<T, const optional<U>&&>` is false.
- is constructible `v<T, const basic_optional<U, Allocator>&&>` is false.
- is convertible `v<const U&, T>` is true.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&, T>` is false.
- is convertible `v<optional<U>&&, T>` is false.
- is convertible `v<optional<U>&&, T>` is false.
- is convertible `v<const optional<U>&, T>` is false.
- is convertible `v<const basic_optional<U, Allocator>&, T>` is false.
- is convertible `v<const optional<U>&&, T>` is false.
- is convertible `v<const basic_optional<U, Allocator>&&, T>` is false.
- is assignable `v<T&, optional<U>&>` is false.
- is assignable `v<T&, basic_optional<U, Allocator>&>` is false.
- is assignable `v<T&, optional<U>&&>` is false.
- is assignable `v<T&, basic_optional<U, Allocator>&&>` is false.
- is assignable `v<T&, const optional<U>&>` is false.
- is assignable `v<T&, const basic_optional<U, Allocator>&>` is false.
- is assignable `v<T&, const optional<U>&&>` is false, and
- is assignable `v<T&, const basic_optional<U, Allocator>&&>` is false

```
template<class... Args>> T& emplace(Args&&...);
```

Constraints: is constructible `v<T, Args&&...>` is true.

Effects: Calls `*this = nullopt`. Then initializes the contained value with `std::forward<Args>(args)...`. If `uses-alloc` is true, initialization is done by `uses-allocator` construction with `allocator alloc`.

Ensures: `*this` contains a value.

Returns: A reference to the new contained value.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: If an exception is thrown during the call to `T`'s constructor, `*this` does not contain a value, and the previous `*val` (if any) has been destroyed.

```
template<class U, class... Args> T& emplace(initializer_list<U>, Args&&...);
```

Constraints: is constructible `v<T, initializer_list<U>&, Args&&...>` is true.

Effects: Calls `*this = nullopt`. Then initializes the contained value with `il` and `std::forward<Args>(args)...`. If `uses-alloc` is true, initialization is done by `uses-allocator` construction with `allocator alloc`.

Ensures: `*this` contains a value.

Returns: A reference to the new contained value.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks: If an exception is thrown during the call to `T`'s constructor, `*this` does not contain a value, and the previous `*val` (if any) has been destroyed.

Swap [basic.optional.swap]

```
void swap(basic_optional& rhs) noexcept(see below);
```

Constraints: Lvalues of type `T` shall be swappable and is move constructible `v<T>` is true.

Preconditions: `L.allocator_traits<Allocator>::propagate_on_container_swap::value` is true or `get_allocator() == s.get_allocator()`.

Effects: If `allocator_traits<Allocator>::propagate_on_container_swap` calls `swap(alloc, rhs.alloc)`. See Table x

Throws: Any exceptions thrown by the operations in the relevant part of Table x.

Remarks: The expression inside `noexcept` is equivalent to: is nothrow move constructible `v<T>` $\wedge$ is nothrow swappable `v<T>`. If any exception is thrown, the results of the expressions `bool(*this)` and `bool(rhs)` remain unchanged. If an exception is thrown during the call to function `swap`, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `swap` for lvalues of `T`. If an exception is thrown during the call to `T`'s move constructor, the state of `tt>*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s move constructor.

Table x — `swap(basic_optional&) effects`

	*this contains a value	*this does not contain a value
--	-------------------------------	---------------------------------------

<u>rhs contains a value</u>	<u>calls <code>swap(*(this), rhs)</code></u>	<u>initializes the contained value with <code>std::move(rhs)</code> as the constructor argument, followed by <code>rhs.val->T::~T()</code>; If <code>uses-alloc</code> is true, initialization is done by uses-allocator construction with allocator <code>alloc</code>. Postcondition is that <code>*this</code> contains a value and <code>rhs</code> does not contain a value</u>
<u>rhs does not contain a value</u>	<u>initializes the contained value with <code>std::move(*(this))</code> as the constructor argument, followed by <code>val->T::~T()</code>; If <code>uses-alloc</code> is true, initialization is done by uses-allocator construction with allocator <code>alloc</code>. Postcondition is that <code>*this</code> does not contain a value and <code>rhs</code> contains a value</u>	<u>no effect</u>

Throws: Any exceptions thrown by the operations in the relevant part of Table x.

Observers [basic.optional.observe]

```
const T* operator->() const;
T* operator->();
  Expects: *this contains a value.
  Returns: val.
  Throws: Nothing.
```

```
const T& operator*() const&;
T& operator*() &;();
  Expects: *this contains a value.
  Returns: *val.
  Throws: Nothing.
```

```
T&& operator*() &&;
const T&& operator*() const&& &;();
  Expects: *this contains a value.
  Effects: Equivalent to: return std::move(*val);.
```

```
explicit operator bool() const noexcept;
  Returns: true if and only if contains a value.
```

```
bool has_value() const noexcept;
  Returns: true if and only if contains a value.
```

```
const T& value() const&;
T& value() &;
  Effects: Equivalent to:
    return bool(*this) ? *val : throw bad optional access();
```

```
T&& value() &&;
const T&& value() const&&;
  Effects: Equivalent to:
    return bool(*this) ? std::move(*val) : throw bad optional access();
```

```
template<class U> T value_or(U&& v) const&;
  Effects: Equivalent to:
    return bool(*this) ? **this : static cast<T>(std::forward<U>(v));
  Remarks: If is copy constructible v<T> && is convertible v<U&&, T is false, the program is ill-formed.
```

```
template<class U> T value_or(U&& v) &&;
  Effects: Equivalent to:
    return bool(*this) ? std::move(**this) : static cast<T>(std::forward<U>(v));
  Remarks: If is move constructible v<T> && is convertible v<U&&, T is false, the program is ill-formed.
```

```
template<class U> T value_or(allocator arg t, const Allocator& a, U&& v) const&;tt>
  Constraints: uses allocator<T, Allocator> is true;
  Effects: if *this contains a value, returns an object of type T initialized by uses-allocator construction with allocator a and **this as the constructor argument. Otherwise, returns an object of type T initialized by uses-allocator construction with allocator a and std::forward<U>(v) as the constructor argument.
  Remarks: If is copy constructible v<T> && is convertible v<U&&, T is false, the program is ill-formed.
```

```
template<class U> T value_or(allocator arg t, const Allocator& a, U&& v) &&;
  Constraints: uses allocator<T, Allocator> is true;
  Effects: if *this contains a value, returns an object of type T initialized by uses-allocator construction with allocator a and std::move(**this) as the constructor argument. Otherwise, returns an object of type T initialized by uses-allocator construction with allocator a and std::forward<U>(v) as the constructor argument.
  Remarks: If is copy constructible v<T> && is convertible v<U&&, T is false, the program is ill-formed.
```

```
allocator type.get_allocator() const noexcept;
  Constraints: uses allocator<T, Allocator> is true;
  Returns: alloc.
  Throws: nothing.
```

Modifiers [basic.optional.mod]

```
void reset() noexcept;
```

Effects: If `*this` contains a value, calls `val->T::~T()` to destroy the contained value; otherwise no effect.

Ensures: `*this` does not contain a value.

Allocator-related traits [basic.optional.traits]

```
template<class T, class Allocator>
struct uses_allocator<T, Allocator> :
    uses_allocator<T, Allocator> { };
```

Relational operators [basic.optional.relops]

```
template<class T, class U, class Allocator> bool operator==(const basic_optional<T, Allocator>&, const basic_optional<U, Allocator>&);
```

```
template<class T, class U, class Allocator> bool operator==(const basic_optional<T, Allocator>&, const optional<U>&);
```

```
template<class T, class U, class Allocator> bool operator==(const optional<T>&, const basic_optional<U, Allocator>&);
Mandates: The expression *x == *y shall be well-formed and its result shall be convertible to bool. [ Note: T need not be Cpp17EqualityComparable. – end note ]
```

Returns: If `bool(x) != bool(y)`, false; otherwise if `bool(x) == false`, true; otherwise `*x == *y`.

```
template<class T, class U, class Allocator> bool operator!=(const basic_optional<T&, Allocator>&, const basic_optional<U, Allocator>&);
```

```
template<class T, class U, class Allocator> bool operator!=(const basic_optional<T, Allocator>&, const optional<U>&);
```

```
template<class T, class U, class Allocator> bool operator!=(const optional<T>&, const basic_optional<U, Allocator>&);
```

Mandates: The expression `*x != *y` shall be well-formed and its result shall be convertible to bool.

Returns: If `bool(x) != bool(y)`, true; otherwise if `bool(x) == false`, false; otherwise `*x != *y`.

```
template<class T, class U, class Allocator> bool operator<(const basic_optional<T, Allocator>&, const basic_optional<U, Allocator>&);
```

```
template<class T, class U, class Allocator> bool operator<(const basic_optional<T, Allocator>&, const optional<U>&);
```

```
template<class T, class U, class Allocator> bool operator<(const optional<T>&, const basic_optional<U, Allocator>&);
```

Mandates: The expression `*x < *y` shall be well-formed and its result shall be convertible to bool.

Returns: If `!y`, false; otherwise if `!x`, true; otherwise `*x < *y`.

```
template<class T, class U, class Allocator> bool operator>(const basic_optional<T, Allocator>&, const basic_optional<U, Allocator>&);
```

```
template<class T, class U, class Allocator> bool operator>(const basic_optional<T, Allocator>&, const optional<U>&);
```

```
template<class T, class U, class Allocator> bool operator>(const optional<T>&, const basic_optional<U, Allocator>&);
```

Mandates: The expression `*x > *y` shall be well-formed and its result shall be convertible to bool.

Returns: If `!x`, false; otherwise if `!y`, true; otherwise `*x > *y`.

```
template<class T, class U, class Allocator> bool operator<=(const basic_optional<T, Allocator>&, const basic_optional<U, Allocator>&);
```

```
template<class T, class U, class Allocator> bool operator<=(const basic_optional<T, Allocator>&, const optional<U>&);
```

```
template<class T, class U, class Allocator> bool operator<=(const optional<T>&, const basic_optional<U, Allocator>&);
```

Mandates: The expression `*x <= *y` shall be well-formed and its result shall be convertible to bool.

Returns: If `!x`, true; otherwise if `!y`, false; otherwise `*x <= *y`.

```
template<class T, class U, class Allocator> bool operator>=(const basic_optional<T, Allocator>&, const basic_optional<U, Allocator>&);
```

```
template<class T, class U, class Allocator> bool operator>=(const basic_optional<T, Allocator>&, const optional<U>&);
```

```
template<class T, class U, class Allocator> bool operator>=(const optional<T>&, const basic_optional<U, Allocator>&);
```

Mandates: The expression `*x >= *y` shall be well-formed and its result shall be convertible to bool.

Returns: If `!y`, true; otherwise if `!x`, false; otherwise `*x >= *y`.

```
template<class T, three way comparable with<T> U, class Allocator>
```

```
compare_three_way result t<T,U>
```

```
operator<=>(const basic_optional<T, Allocator>&, const basic_optional<U, Allocator>&);
```

```
template<class T, three way comparable with<T> U, class Allocator>
```

```
compare_three_way result t<T,U>
```

```
operator<=>(const basic_optional<T, Allocator>&, const optional<U>&);
```

```
template<class T, three way comparable with<T> U, class Allocator>
```

```
compare_three_way result t<T,U>
```

```
operator<=>(const optional<T>&, const basic_optional<U, Allocator>&);
```

Returns: If `x && y`, `*x <=> *y`; otherwise `bool(x) <=> bool(y)`.

Comparison with nullopt [basic.optional.nullopts]

```
template<class T, class Allocator> bool operator==(const basic_optional<T, Allocator>&, nullopt t) noexcept;
```

Returns: `!x`.

```
template<class T, class Allocator> strong_ordering operator<=>(const basic_optional<T, Allocator>&, nullopt t) noexcept;
```

Returns: `bool(x) <=> false`.

Comparison with T [basic.optional.comp.with.t]

```
template<class T, class U, class Allocator> bool operator==(const basic_optional<T, Allocator>& x, const U& v);
```

Mandates: The expression `*x == v` shall be well-formed and its result shall be convertible to bool. [Note: T need not be Cpp17EqualityComparable. – end note]

Effects: Equivalent to: `return bool(x) ? *x == v : false;`

```
template<class T, class U, class Allocator> bool operator==(const T& t, const basic_optional<U, Allocator>& v);
```

Mandates: The expression `x == *v` shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: `return bool(v) ? x == *v : false;`

```
template<class T, class U, class Allocator> bool operator!=(const basic_optional<T, Allocator>& u, const U& v);
```

Mandates: The expression `*x != v` shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: `return bool(x) ? *x != v : true;`

```
template<class T, class U, class Allocator> bool operator!=(const T& t, const basic_optional<U, Allocator>& v);
```

Mandates: The expression `x != *v` shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: `return bool(v) ? x != *v : true;`

`template<class T, class U, class Allocator> bool operator<(const basic_optional<T, Allocator>&, const U&);`
Mandates: The expression `*x < v` shall be well-formed and its result shall be convertible to bool.
Effects: Equivalent to: `return bool(x) ? *x < v : true;`

`template<class T, class U, class Allocator> bool operator<(const T& t, const basic_optional<U, Allocator>& v);`
Mandates: The expression `x < *v` shall be well-formed and its result shall be convertible to bool.
Effects: Equivalent to: `return bool(v) ? x < *v : false;`

`template<class T, class U, class Allocator> bool operator>(const basic_optional<T, Allocator>&, const U&);`
Mandates: The expression `*x > v` shall be well-formed and its result shall be convertible to bool.
Effects: Equivalent to: `return bool(x) ? *x > v : false;`

`template<class T, class U, class Allocator> bool operator>(const T& t, const basic_optional<U, Allocator>& v);`
Mandates: The expression `x > *v` shall be well-formed and its result shall be convertible to bool.
Effects: Equivalent to: `return bool(v) ? x > *v : true;`

`template<class T, class U, class Allocator> bool operator<=(const basic_optional<T, Allocator>&, const U&);`
Mandates: The expression `*x <= v` shall be well-formed and its result shall be convertible to bool.
Effects: Equivalent to: `return bool(x) ? *x <= v : true;`

`template<class T, class U, class Allocator> bool operator<=(const T& t, const basic_optional<U, Allocator>& v);`
Mandates: The expression `x <= *v` shall be well-formed and its result shall be convertible to bool.
Effects: Equivalent to: `return bool(v) ? x <= *v : false;`

`template<class T, class U, class Allocator> bool operator>=(const basic_optional<T>&, const U&);`
Mandates: The expression `*x >= v` shall be well-formed and its result shall be convertible to bool.
Effects: Equivalent to: `return bool(x) ? *x >= v : false;`

`template<class T, class U, class Allocator> bool operator>=(const T& t, const basic_optional<U, Allocator>& v);`
Mandates: The expression `x >= *v` shall be well-formed and its result shall be convertible to bool.
Effects: Equivalent to: `return bool(v) ? x >= *v : true;`

`template<class T, three way comparable with<T> U, class Allocator>`
`compare_three_way_result t<T,U>`
`operator<=>(const basic_optional<T, Allocator>& x, const U&v);`
Effects: Equivalent to: `return bool(x) ? *x >= v : strong_ordering::less ;`

Specialized algorithms [basic.optional.specalg]

`template<class T, class Allocator> void swap(basic_optional<T, Allocator>& x, basic_optional<T, Allocator>& y);`
Effects: if `x.get_allocator() == y.get_allocator()`, calls `x.swap(y)`. Otherwise equivalent to
`basic_optional<T, Allocator> futureX(allocator arg t, x.get_allocator(), y);`
`basic_optional<T, Allocator> futureY(allocator arg t, y.get_allocator(), x);`
`futureX.swap(x);`
`futureY.swap(y);`

`template<class T> constexpr basic_optional<decay_t<T>> make_basic_optional(T&&);`
Returns: `optional<decay_t<T>>(std::forward<T>(v))`.
Remarks: Specialization of this function template shall be a constexpr function if `uses-alloc::value` is false.

`template<class T> constexpr basic_optional<T> make_basic_optional();`
Effects: Equivalent to: `return basic_optional<T>(in place)`.
Remarks: Specialization of this function template shall be a constexpr function if `uses-alloc::value` is false.

`template<class T, class Arg0, class... Args> constexpr basic_optional<T> make_basic_optional(Arg0&& arg0, Args&&... args);`
Effects: Equivalent to: `return basic_optional<T>(in place, std::forward<Arg0>(arg0), std::forward<Args>(args)...)`.
Constraints: `!is_same<typename remove_cvref_t<Arg0>, allocator arg t>`
Remarks: Specialization of this function template shall be a constexpr function if `uses-alloc::value` is false.

`template<class T, class... Args> constexpr basic_optional<T> make_basic_optional(Args&&... args);`
Effects: Equivalent to: `return basic_optional<T>(in place, std::forward<Args>(args)...)`.
Remarks: Specialization of this function template shall be a constexpr function if `uses-alloc::value` is false.

`template<class T, class U, class... Args> basic_optional<T>`
`constexpr make_basic_optional(initializer_list<U> il, Args&&... args);`
Effects: Equivalent to: `return basic_optional<T>(in place, il, std::forward<Args>(args)...)`.
Remarks: Specialization of this function template shall be a constexpr function if `uses-alloc::value` is false.

`template<class T, class Allocator> basic_optional<decay_t<T>, Allocator>`
`make_basic_optional(allocator arg t, const Allocator& a, T&&);`
Mandates: `uses_allocator<T, Allocator>` is true.
Returns: `basic_optional<decay_t<T>, Allocator>(allocator arg, a, in place, std::forward<T>(v))`.

`template<class T, class Allocator, class... Args> basic_optional<T, Allocator>`
`make_basic_optional(allocator arg t, const Allocator& a, Args&&... args);`
Mandates: `uses_allocator<T, Allocator>` is true.
Effects: Equivalent to: `return basic_optional<T, Allocator>(allocator arg, a, in place, std::forward<Args>(args)...)`.

`template<class T, class Allocator, class U, class... Args> basic_optional<T, Allocator>`
`make_basic_optional(allocator arg t, const Allocator& a, initializer_list<U> il, Args&&... args);`
Mandates: `uses_allocator<T, Allocator>` is true.
Effects: Equivalent to: `return basic_optional<T, Allocator>(allocator arg, a, in place, il, std::forward<Args>(args)...)`.

Hash support [basic.optional.hash]

`template<class T, class Allocator> struct hash<basic_optional<T, Allocator>>;`
The specialization `hash<basic_optional<T, Allocator>>` is enabled ([unord.hash]) if and only if `hash<remove_const_t<T>>` is enabled. When enabled, for an object `o` of type `basic_optional<T, Allocator>`, if `bool(o) == true`, then `hash<basic_optional<T, Allocator>>(o)` shall evaluate to the same value as `hash<remove_const_t<T>>(*o)`; otherwise it evaluates to an unspecified value. The member functions are not guaranteed to be noexcept.

Specialized algorithms [basic.pmroptional.specalg]

template<class T> constexpr optional<decay t<T>> make optional(T&&);
Returns: optional<decay t<T>>(std::forward<T>(v)).
Remarks: Specialization of this function template shall be a constexpr function if uses-alloc::value is false.

template<class T> constexpr optional<T> make optional();
Effects: Equivalent to: return optional<T>(in place).
Remarks: Specialization of this function template shall be a constexpr function if uses-alloc::value is false.

template<class T, class Arg0, class... Args> constexpr optional<T> make optional(Arg0&& arg0, Args&&... args);
Effects: Equivalent to: return optional<T>(in place, std::forward<Arg0>(arg0), std::forward<Args>(args)...).
Constraints: !is same< typename remove cvref t<ARG0>, allocator arg t>
Remarks: Specialization of this function template shall be a constexpr function if uses-alloc::value is false.

template<class T, class... Args> constexpr optional<T> make optional(Args&&... args);
Effects: Equivalent to: return optional<T>(in place, std::forward<Args>(args)...).
Remarks: Specialization of this function template shall be a constexpr function if uses-alloc::value is false.

template<class T, class U, class... Args> optional<T>
constexpr make optional(initializer list<U> il, Args&&... args);
Effects: Equivalent to: return optional<T>(in place, il, std::forward<Args>(args)...).
Remarks: Specialization of this function template shall be a constexpr function if uses-alloc::value is false.

template<class T> optional<decay t<T>>
make optional(allocator arg t, const polymorphic allocator<>& a, T&&);
Mandates: uses allocator<T, polymorphic allocator<>> is true;
Returns: optional<decay t<T>>(allocator arg, a, in place, std::forward<T>(v)).

template<class T, class... Args> optional<T>
make optional(allocator arg t, const polymorphic allocator<>& a, Args&&... args);
Mandates: uses allocator<T, polymorphic allocator<>> is true;
Effects: Equivalent to: return optional<T>(allocator arg, a, in place, std::forward<Args>(args)...).

template<class T, class U, class... Args> optional<T>
make optional(allocator arg t, const polymorphic allocator<>& a, initializer list<U> il, Args&&... args);
Mandates: uses allocator<T, polymorphic allocator<>> is true;
Effects: Equivalent to: return optional<T>(allocator arg, a, in place, il, std::forward<Args>(args)...).